

Exercício 03: Automação e Condições – UFCD 10806

Diretório de entrega:	ex03/
Ficheiros a entregar:	backup_dados.sh (Linux/Mac) ou backup_dados.ps1 (Windows)
Funções permitidas:	Comandos nativos da consola e estruturas de controlo abordadas no Bloco 3 (if, else, argumentos e variáveis)

Descrição

Na Ciência de Dados, o teu *dataset* original é sagrado. Antes de corrermos algoritmos pesados ou scripts de limpeza que possam alterar ficheiros acidentalmente, é boa prática criar rotinas automáticas de *backup*.

O teu objetivo é criar um script interativo que receba o nome de um ficheiro de dados, verifique de forma inteligente se ele existe e crie uma cópia de segurança.

O teu script deve executar os seguintes passos e regras lógicas:

1. **Captura de Argumentos:** O script deve receber o nome do ficheiro a copiar através do primeiro argumento da linha de comandos (ex: \$1 em Bash).
2. **Validação de Argumentos:** Se o utilizador se esquecer de passar um argumento, o script deve avisá-lo, imprimindo no ecrã a mensagem exata: Uso correto: `./backup_dados.sh <nome_do_ficheiro>`, e terminar imediatamente.
3. **Validação de Ficheiro:** Se o argumento for passado, o script deve testar se o ficheiro realmente existe no sistema.
 - **Em caso de erro (não existe):** Imprimir Erro: O ficheiro [NOME_DO_FICHEIRO] não foi encontrado.
 - **Em caso de sucesso (existe):** Copiar o ficheiro original adicionando-lhe a extensão `.bak` no final (exemplo: se o original for `dados.csv`, a cópia será `dados.csv.bak`). Após copiar, imprimir Backup de [NOME_DO_FICHEIRO] criado com sucesso!.

(Nota: Substitui [NOME_DO_FICHEIRO] pelo nome real do ficheiro passado pelo utilizador).

Exemplo de Validação (Em ambiente Linux/Mac)

Teste 1: Esquecer o argumento

```
$> ./backup_dados.sh
```

Uso correto: ./backup_dados.sh <nome_do_ficheiro>

```
$>
```

Teste 2: Passar um ficheiro que não existe

```
$> ./backup_dados.sh dataset_fantasma.csv
```

Erro: O ficheiro dataset_fantasma.csv não foi encontrado.

```
$>
```

Teste 3: Passar um ficheiro válido (sucesso)

```
$> touch dataset_oficial_2026.csv
```

```
$> ./backup_dados.sh dataset_oficial_2026.csv
```

Backup de dataset_oficial_2026.csv criado com sucesso!

```
$> ls -l
```

```
-rwxr-xr-x 1 utilizador utilizador 0 Oct 2 15:30 backup_dados.sh
```

```
-rw-r--r-- 1 utilizador utilizador 0 Oct 2 15:31 dataset_oficial_2026.csv
```

```
-rw-r--r-- 1 utilizador utilizador 0 Oct 2 15:32 dataset_oficial_2026.csv.bak
```

```
$>
```

Exemplo de Validação (Em ambiente Windows)

Teste 1, 2 e 3 (Fluxo no PowerShell)

```
PS C:\ex04> .\backup_dados.ps1
```

Uso correto: .\backup_dados.ps1 <nome_do_ficheiro>

```
PS C:\ex04> .\backup_dados.ps1 dataset_fantasma.csv
```

Erro: O ficheiro dataset_fantasma.csv não foi encontrado.

```
PS C:\ex04> New-Item dataset_oficial_2026.csv -ItemType File
```

```
PS C:\ex04> .\backup_dados.ps1 dataset_oficial_2026.csv
```

Backup de dataset_oficial_2026.csv criado com sucesso!

```
PS C:\ex04>
```

*Dicas: > * Para Bash (Linux/Mac):* Revê os slides sobre "Argumentos de Linha de Comandos", o operador -z para verificar se variáveis estão vazias e o operador -f para testar se ficheiros existem.

- *Para PowerShell (Windows):* Usa o array \$args[0] para o primeiro argumento e o Test-Path para validar a existência do ficheiro.